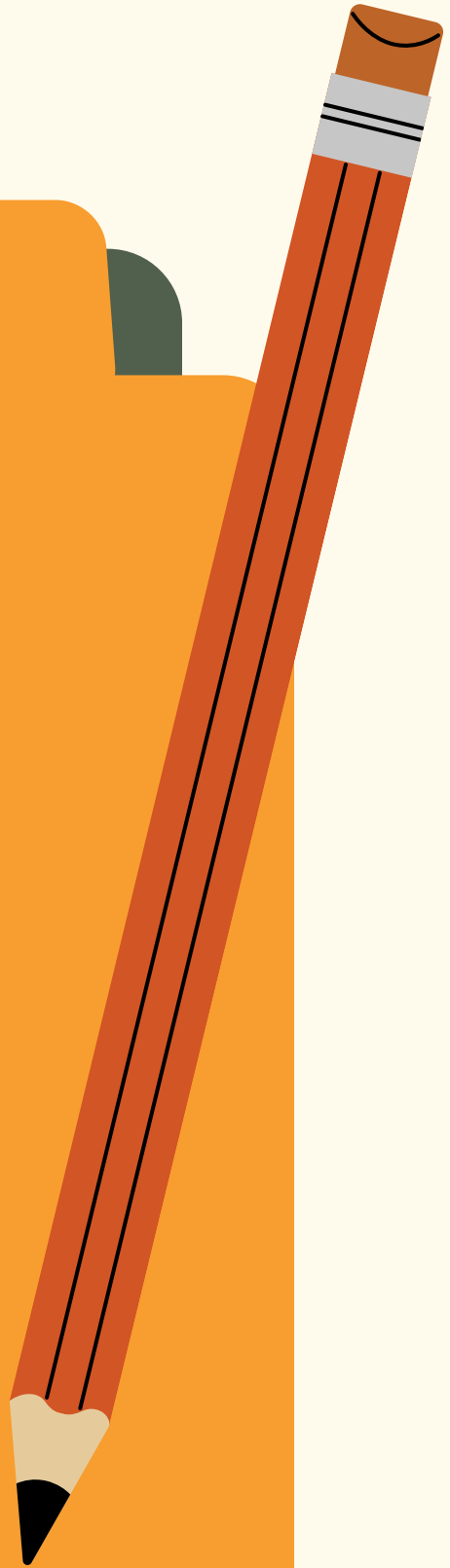
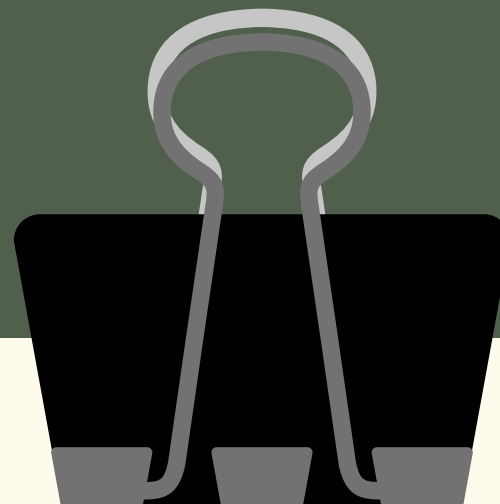


From numbers to AI

PROGRAMMING

파이썬





질문있나요?

<https://discord.gg/mQa86T9NU2>

cm@playidealab.com 또는 lcmin@yonsei.ac.kr

모르는건 당연한거지만, 모르는채로 있는건 선택이지요

질문하는 과정에서 깨닫기도 합니다



파이썬 기초

자료형

가변객체

이창민



자료형 II

가변객체

가변객체

- **List** : 순서를 가지는 값의 나열
- **Dictionary** : key와 그에 대응되는 value
- **Set** : 중복 없는 key의 모음

List

원소와 순서의 관리 (지지고 볶고~)

List

- List 만들기
 - 기호 [] (default)
 - 키워드 list

여러줄 걸쳐서도 가능

```
a = ['가',  
     '나',  
     '다']  
print(a)
```

['가', '나', '다']

```
[4] 1 x = []  
2 print(x,type(x))  
3 x = list()  
4 print(x,type(x))  
5 x = [1]  
6 print(x,type(x))  
7 x = [1,2.0,True,None,"string",(1,3)]  
8 print(x,type(x))
```

```
⇒ [] <class 'list'>  
[] <class 'list'>  
[1] <class 'list'>  
[1, 2.0, True, None, 'string', (1, 3)] <class 'list'>
```

[] 기호를 통해 리스트로 타입이 정해짐

무엇이든 보관할 수 있다

```
[57] 1 x = list('0123456789')  
2 print(x, type(x), len(x))  
3 print(x[-1])  
4 print(x[0:-1])  
5 print(x[2:8:2])  
6 x[0] = 99  
7 print(x)
```

```
⇒ ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9'] <class 'list'> 10  
9  
['0', '1', '2', '3', '4', '5', '6', '7', '8']  
['2', '4', '6']  
[99, '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

길이를 구할 수 있고

인덱싱, 슬라이싱이 가능하다

위치로 접근하여 값을 수정할 수 있다(가변객체)

List

list()보다 [] 가 더 빠르다!

% :: magic command 참조

```
%timeit my_list_1 = []  
%timeit my_list_1 = list()
```

The slowest run took 30.72 times longer than the fastest. This could mean that an intermediate result is being cached.
100000000 loops, best of 5: 34.9 ns per loop
The slowest run took 11.32 times longer than the fastest. This could mean that an intermediate result is being cached.
100000000 loops, best of 5: 104 ns per loop

```
from dis import dis  
dis("[]")  
dis("list()")
```

disassembler

실제로 어떤 명령어가 호출되는지 어셈블러를 추적함.

1	0 BUILD_LIST	0
	2 RETURN_VALUE	
1	0 LOAD_NAME	0 (list)
	2 CALL_FUNCTION	0
	4 RETURN_VALUE	

왜?

컴퓨터가 list()라는 명령어가 어떤 건지 찾는 시간이 든다!

[] 가 빠르고 간편하다

같은 이유로 dictionary도 dict()보단 {}를 쓰자

List

Type cast

iterable한 객체만 list로 변환가능함

string은 iterable함: 변환가능

다시 string으로 바뀌니까 기호가?

tuple은 list로 바꿀수 있다!
반대도 가능하다!

```
1 x = list(1)
```

Traceback (most recent call last)
<ipython-input-12-6df1b354d64e> in <cell line: 0>()
----> 1 x = list(1)

TypeError: 'int' object is not iterable

```
[5] 1 x = "hello world"
2 print(x)
3 y = list(x)
4 print(y)
5 z = str(y)
6 print(z)
```

hello world
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']

```
1 dir("")
```

['__init_subclass__',
 'iter',
 'le',
 'len']

```
1 print(z[0:2])
```

['h', 'e']

```
[15] 1 x = (1,2,3,)
2 print(x,type(x))
3 y = list(x)
4 print(y,type(y))
5 z = tuple(y)
6 print(z,type(z))
```

(1, 2, 3) <class 'tuple'>
[1, 2, 3] <class 'list'>
(1, 2, 3) <class 'tuple'>

```
[18] 1 x = (1,2,3,)
2 y = list(x)
3 y[0] = 99
4 x = tuple(y)
5 print(x,type(x))
```

(99, 2, 3) <class 'tuple'>

list로 바꿔서 수정

✓
0초

[58]1dir([])

↗

['__add__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__rmul__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'append',
 'clear',
 'copy',
 'count',
 'extend',
 'index',
 'insert',
 'pop',
 'remove',
 'reverse',
 'sort']

✓
0초

▶

1x = [0,1,2] + [3,4,5]
2print(x)
3x = 3*x
4print(x)

↗

[0, 1, 2, 3, 4, 5]
[0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5]

더하거나 곱하거나!

Ⓢ
0초

▶

1x = [0,1,2,3,4,5]
2del x[2]
3print(x)
4del x
5print(x)

↗

[0, 1, 3, 4, 5]

Traceback (most recent call last)
 <ipython-input-62-a5669b077668> in <cell line: 0>()
 3 print(x)
 4 del x
----> 5 print(x)

NameError: name 'x' is not defined

원소를 지우거나 전체 리스트를 지우거나

✓
0초

[69]

1print([1,2,3]==[1,2,3])
2print([1, 2, 3] < [1, 2, 4])
3print([1, 2] < [1, 2, 0])
4print([1, 2, 3] > [1, 2])
5print([1, 2, 3] < [1, 2, 3])

↗

True
True
True
True
False

True
True → 세 번째 원소가 3 < 4
True → 앞의 두 원소가 같고, 더 짧아서 작음
True → 앞의 두 원소가 같고, 더 긴 쪽이 큼

사전식 비교

✓
0초

[63]

1print(1 in [1,2,3])
2print([1,2] in [1,2,3])

↗

True
False

특정 원소를 가지고 있는지 체크

List

List 클래스 메소드

- 요소 추가

```
a = [1, 2, 3, 4]
a.append(5)
a
```

[1, 2, 3, 4, 5]

a+= [5]

- 리스트 연결

```
a = [1, 2, 3]
a.extend([4, 5, 6])
a
```

[1, 2, 3, 4, 5, 6]

a+= [4, 5, 6]

- 차이점

```
a = [1, 2, 3]
```

```
a.append([5, 6])
a
```

[1, 2, 3, [5, 6]]

```
a.extend([5, 6])
a
```

[1, 2, 3, 5, 6]

```
a = [1, 2, 3, ['a', 'b', 'c']]
```

```
a[-1][0]
```

'a'

```
a = [1, 2, 3, ('a', 'b', 'c')]
a[-1][0]
```

'a' # 튜플도 마찬가지로

```
a = [1, 2, ['a', 'b', ['Life', 'is']]]
a[2][2][0]
```

'Life'

Tuple, List 예제

```
a = [1, 2, 3]
a[0] + a[2]
```

4

```
a = [1, 2, 3, ['a', 'b', 'c'], 4, 5]
print(a[2:5])
print(a[3][:2])
```

```
[3, ['a', 'b', 'c'], 4]
['a', 'b']
```

```
a = [1, 2, 3]
a[2] + 'hi'
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-19-5d86398f5cfd> in <module>()
```

```
      1 a = [1, 2, 3]
----> 2 a[2] + 'hi'
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
str(a[2]) + 'hi'
```

```
'3hi'
```

List

List 클래스 메소드

```
my_list = ['a', 'b', 'd', 'a', 'c', 'a']
```

- 특정 값의 인덱스 번호 알려줌

```
my_list.index('b')
```

1

- 특정 값이 몇 개인지 세어보자

```
my_list.count('a')
```

3

- 원하는 자리에 요소를 끼워 넣어보자

```
a = [71, 72, 73, 74]
a.insert(2, 77)
a
```

2번째에 77을 넣어라

```
[71, 72, 77, 73, 74]
```

- 문자열 split

```
s = "Nice to meet you guys"
a = s.split()
a
```

```
['Nice', 'to', 'meet', 'you', 'guys']
```

```
a = [1, 2, 3]
a.index(3)
```

2

```
a.index(0)
```

```
-----
ValueError                                Trace
<ipython-input-31-affdffa288f1> in <module>()
----> 1 a.index(0)
```

```
ValueError: 0 is not in list
```

List

List 클래스 메소드

- 값으로 제거

```
a = [1, 2, 3, 4, 5, 6]
a.remove(4)
a          # 요소 지정
[1, 2, 3, 5, 6]
```

- index로 제거

```
a = [1, 2, 3, 4, 5, 6]
del a[3]
a          # 인덱스 번호 지정
[1, 2, 3, 5, 6]
```

- index로 값을 추출(비복원)

```
a = [1, 2, 3]
a.pop(1)

2    # 1번째 요소

a    # 를 돌려주고 요소 삭제
[1, 3]
```

List

정렬

- 순서를 반대로

```
a = [1, 10, 5, 7, 6]
a.reverse()
a
```

[6, 7, 5, 10, 1]

- 정렬 다른 방법

```
a = [1, 10, 5, 7, 6]
sorted(a)
```

[1, 5, 6, 7, 10]

```
sorted(a, reverse=True)
```

[10, 7, 6, 5, 1]

```
a = [(10,5), (12,1), (42,16), (2,13)]
print(type(a))
b = sorted(a, key=lambda x:x[1])
print(b)
```

key를 선언할 수 있음

```
<class 'list'>
[(12, 1), (10, 5), (2, 13), (42, 16)]
```

- 오름차순으로

```
a = [1, 10, 5, 7, 6]
a.sort()
a
```

[1, 5, 6, 7, 10]

- 내림차순으로

```
a = [1, 10, 5, 7, 6]
a.sort(reverse=True)
a
```

[10, 7, 6, 5, 1]

- 글자 길이 기준으로 정렬

```
a = ['나는', '파이썬을', '잘하고', '싶다']
a.sort(key=len)
a
```

['나는', '싶다', '잘하고', '파이썬을']

List

- 요소를 삭제하거나 변경할 수 있음
- mutable(가변성) iterable(순차적인)
- list + list 형식으로 서로 붙임 가능
- 서로 다른 type의 요소들을 담을 수 있음

```
l = []           // 리스트 선언
l = list()       // 리스트 선언
list(seq)        // 리스트로 변환
l.append(1)       // list에 값 1을 요소로 추가한다.
l.insert(2, 'data') // list의 두 번째 요소에 'data'를 삽입한다.
del l[index]      // index 기반 삭제
l.remove(삭제할값) // value 기반 삭제
l.pop()           // list의 tail 삭제 및 반환
l.pop(0)          // list의 head 삭제 및 반환
l.extend(new_list) // list l과 new_list를 병합한다.
l += new_list      // list l과 new_list를 병합한다.
b = l.copy()       // list l의 복사값을 b에 대입
b = list(l)        // list l의 복사값을 b에 대입
b = l[:]           // list l의 복사값을 b에 대입
l.index('data')    // list 요소 중 'data'의 인덱스 반환
'data' in l        // list에 'data' 요소가 있는지 True, False 반환
l.count('data')    // 'data' 개수 확인
", ".join(l)       // ,를 기준으로 list를 문자열로 만든다.
sorted(l)          // 정렬된 복사본을 반환
l.sort()           // list 자체를 정렬한다.
len(l)             // list 개수
max(l)             // 요소 중 최대값
min(l)             // 요소 중 최소값
```

Tuple, List 예제

```
firstname, lastname = 'Changmin', 'Lee'
print(firstname)
print(lastname)
print(lastname, '-', firstname)
```

Changmin
Lee
Lee - Changmin

```
a = (38, 21, 53, 62, 19)
print(min(a))    # 최솟값
print(max(a))    # 최댓값
print(sum(a))    # 모든 요소 합
```

19
62
193

reversed

```
name = ['John', 'Alice', 'Lisa']
for i, k in enumerate(name): # index, key값 반환
    print(f'{i+1}번 : {k}')
```

1번 : John
2번 : Alice
3번 : Lisa

```
my_list = ['AB', 10, False]
print(my_list[2])
```

False # 가변객체

```
my_tuple = ('AB', 10, False)
print(my_tuple[2])
```

False # 불변객체

```
my_list[2] = True
print(my_list)
```

['AB', 10, True]

```
my_tuple[2] = True
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-6-74704114a58a> in <module>()
----> 1 my_tuple[2] = True
```

TypeError: 'tuple' object does not support item assignment

Tuple, List 예제

```
a = (38, 21, 53, 62, 19)
a.remove(21)
a
```

```
-----
AttributeError                                Traceback (most
<ipython-input-26-6b9dace5802a> in <module>()
      1 a = (38, 21, 53, 62, 19)
----> 2 a.remove(21)
      3 a
```

AttributeError: 'tuple' object has no attribute 'remove'

SEARCH STACK OVERFLOW

```
b = list(a)
b.remove(21)
b
```

```
[38, 53, 62, 19]
```

```
a = ([1,2], [3,4,5])
print(type(a))
print(type(a[0]))
```

```
<class 'tuple'>
<class 'list'>
```

```
a[1].append(6)           # 리스트는 가변객체
a

([1, 2], [3, 4, 5, 6])
```

```
a.append(6)             # 튜플은 불변객체
a
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-24-7379f31eb17f> in <module>()
----> 1 a.append(6)
      2 a
```

AttributeError: 'tuple' object has no attribute 'append'

Tuple, List 예제

```
list2 = [3, 1, 9, 4, 2, 8]
```

```
list2.sort()  
list2
```

```
[1, 2, 3, 4, 8, 9]
```

```
list2.sort(reverse=True)  
list2
```

```
[9, 8, 4, 3, 2, 1]
```

```
list3 = ['p', 'f', 'x', 'a']
```

```
list3.sort()  
list3
```

```
['a', 'f', 'p', 'x']
```

```
list3.sort(reverse=True)  
list3
```

```
['x', 'p', 'f', 'a']
```

```
list4 = [1, 5, 2, 'a', 'b']  
list4.sort()
```

문자, 숫자 섞여 있으면 sort() 적용시 TypeError

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-55-3694ec9d3edf> in <module>()  
      1 list4 = [1, 5, 2, 'a', 'b']  
----> 2 list4.sort()
```

TypeError: '<' not supported between instances of 'str' and 'int'

```
my_list = [1, 2, 'a', 'b']  
my_list.append('c')  
my_list
```

```
[1, 2, 'a', 'b', 'c']
```

```
my_list.extend([3, 4, 5])  
my_list
```

```
[1, 2, 'a', 'b', 'c', 3, 4, 5]
```

```
my_list.insert(0, 'a')  
my_list
```

```
['a', 1, 2, 'a', 'b', 'c', 3, 4, 5]
```

```
my_list.count('a')
```

```
2
```

```
my_list.index('b')
```

```
4
```

Tuple, List 예제

List Comprehension (맛보기)

반복문, 조건문 참고

```
# List Comprehension
my_list = [n ** 2 for n in range(10) if n % 3 == 0]
print(my_list)
```

[0, 9, 36, 81] # 0~9까지의 수 중 3의 배수인 수를 제공한 값들

```
a = [i * j for j in range(2, 10)
      for i in range(1, 10)] # for, if 여러 번 사용 가능
print(a)                   # 구구단 결과값들 출력
```

[2, 4, 6, 8, 10, 12, 14, 16, 18, 3, 6, 9, 12, 15, 18, 21, 24, 27, 4, 8, 12, 16, 20, 24, 28, 32, 36, 5, 10, 15, 20]

Tuple도 됨

```
tuple(i for i in range(10))
```

(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

```
[i for i in range(10)]
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

빈칸 채우기

Tuple과 List 예제

- 튜플, 리스트 슬라이싱
- 리스트는 튜플과 다르게 요소를 바꿀 수 있다.

(1)

```
a = ('a', ('s', 'd'), ('f', 'g', 'h'))  
print( )
```


g

(2)

```
a = [4, 6, 2, 7, 4]  
a  
[2, 4, 4, 6, 7]
```

(3)

```
a = ([1, 2, 3])  
type(a)
```

```
a = ([1, 2, 3],)  
type(a)
```


(4)

```
my_str = "Hi-I-am-a-robot"  
my_list =  
print(my_list)
```


['Hi', 'I', 'am', 'a', 'robot']

문자열 'I want to sleep.' 를
띄어쓰기를 기준으로 나눠서 리스트로 변환하고,
sleep 을 다른 동사로 바꿔 보시다.

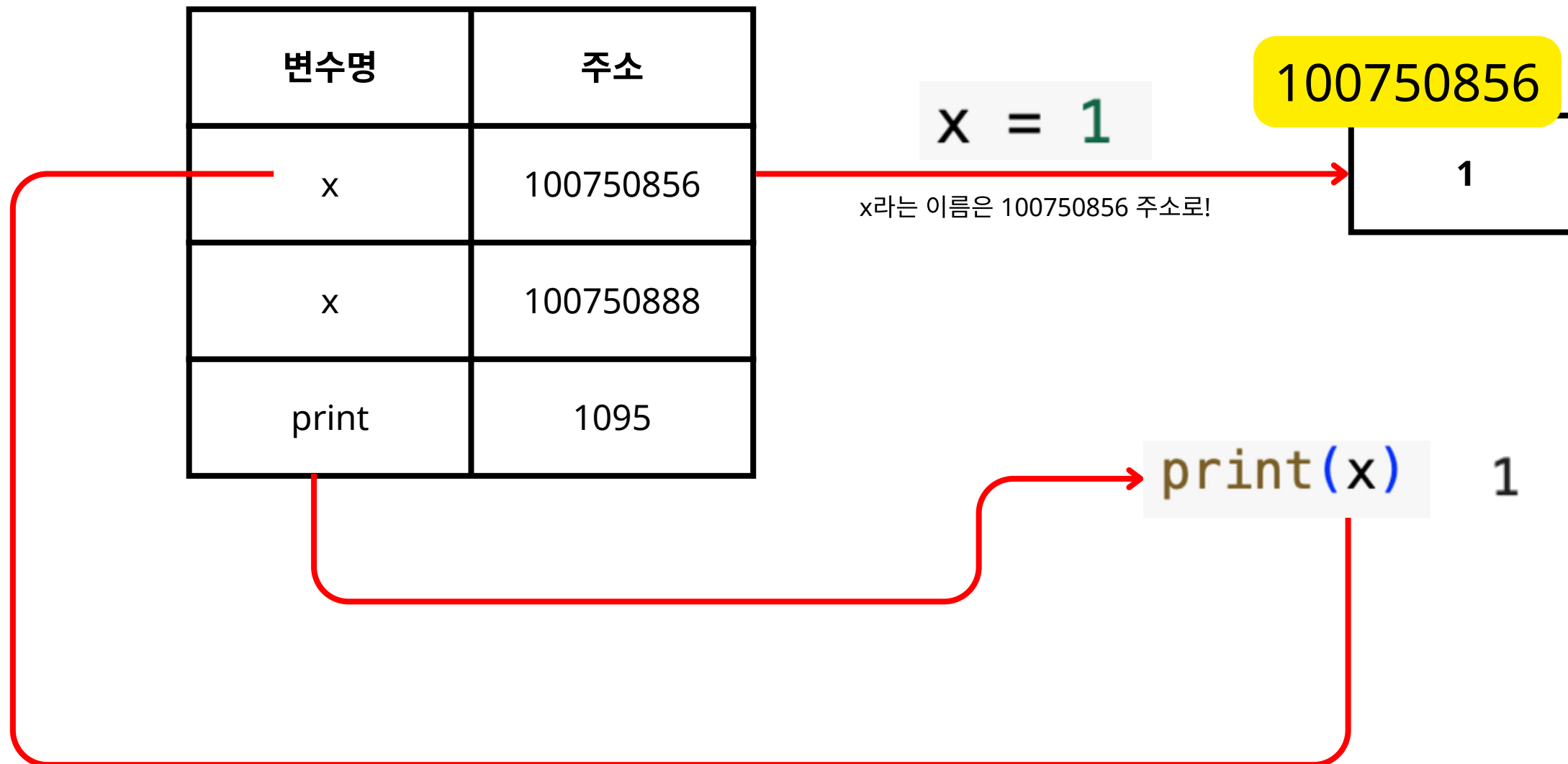
```
my_str = 'I want to sleep.'  
my_str_list =  
print(my_str_list)  
= 'rest'  
print(my_str_list)
```

```
['I', 'want', 'to', 'sleep.']  
['I', 'want', 'to', 'rest']
```

(5)

불변객체: immutable

Call by value



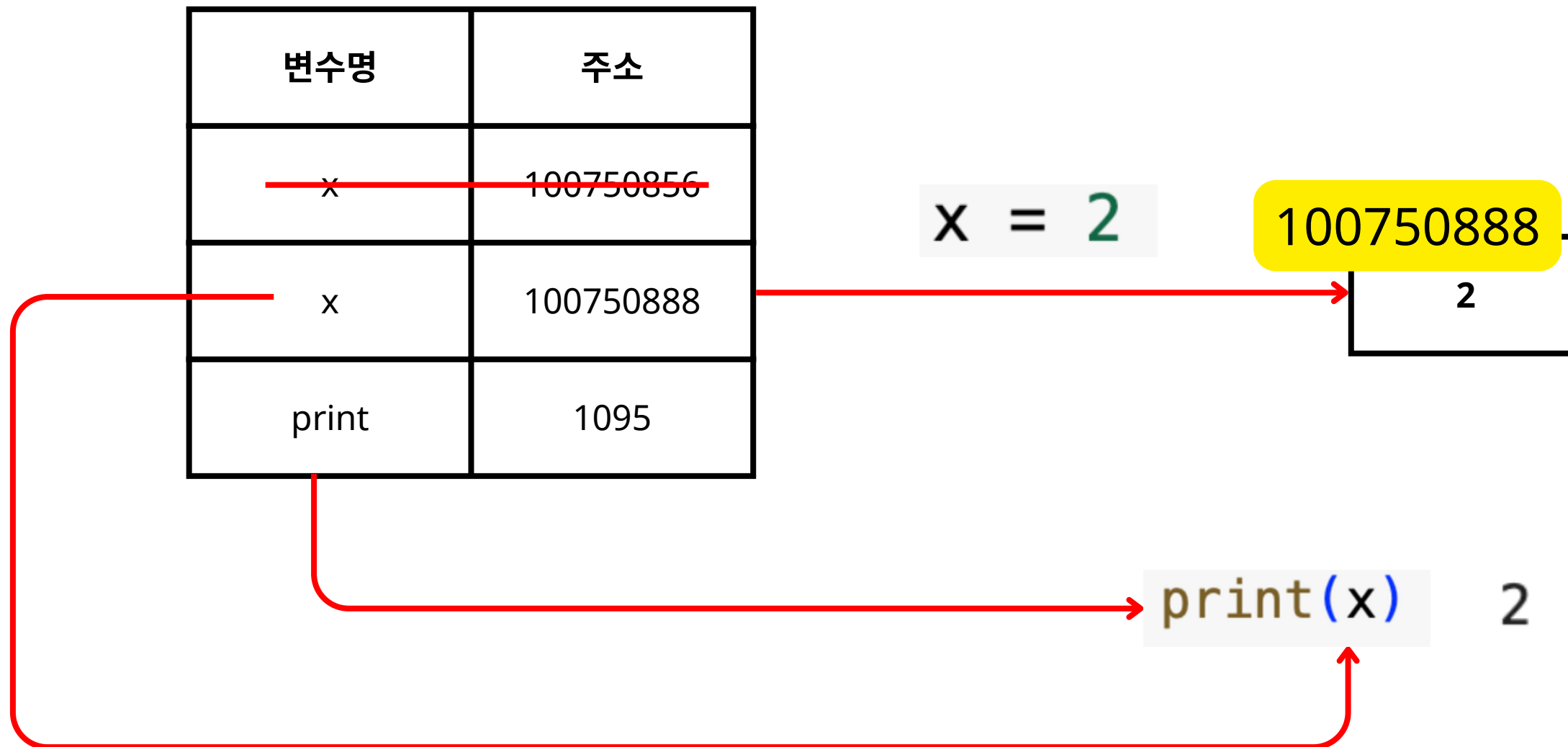
✓
0초

```
[2]      1  x = 1
        2  print(x, id(x))
        3  x = 2
        4  print(x, id(x))
```

```
⇒ 1 10750856
   2 10750888
```

불변객체: immutable

Call by value



✓
0초

```
[2]      1  x = 1
        2  print(x, id(x))
        3  x = 2
        4  print(x, id(x))
```

```
⇒ 1 10750856
   2 10750888
```

가변객체인 List: mutable

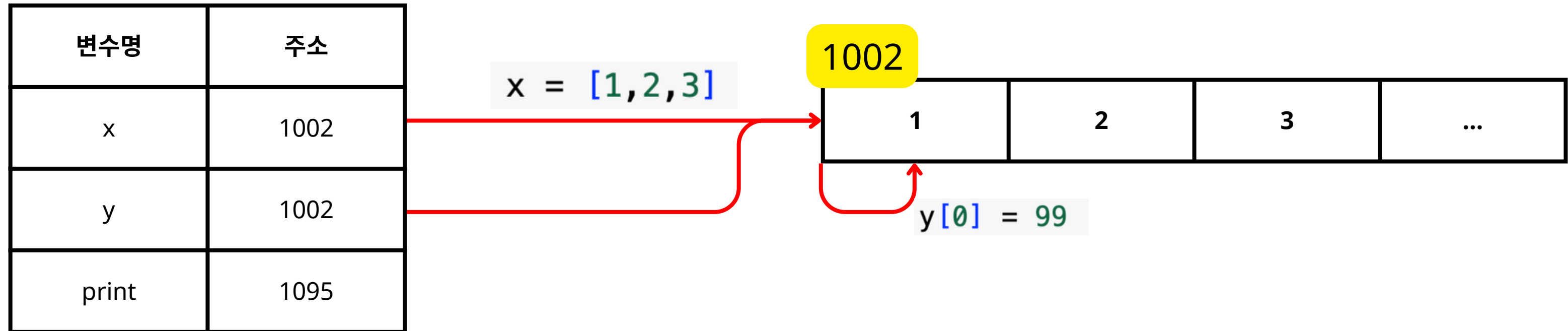
Call by reference

✓
0초



```
1 x = [1, 2, 3]
2 y = x
3 y[0] = 99
4 print(x)
```

⇒ [99, 2, 3]



가변객체인 List

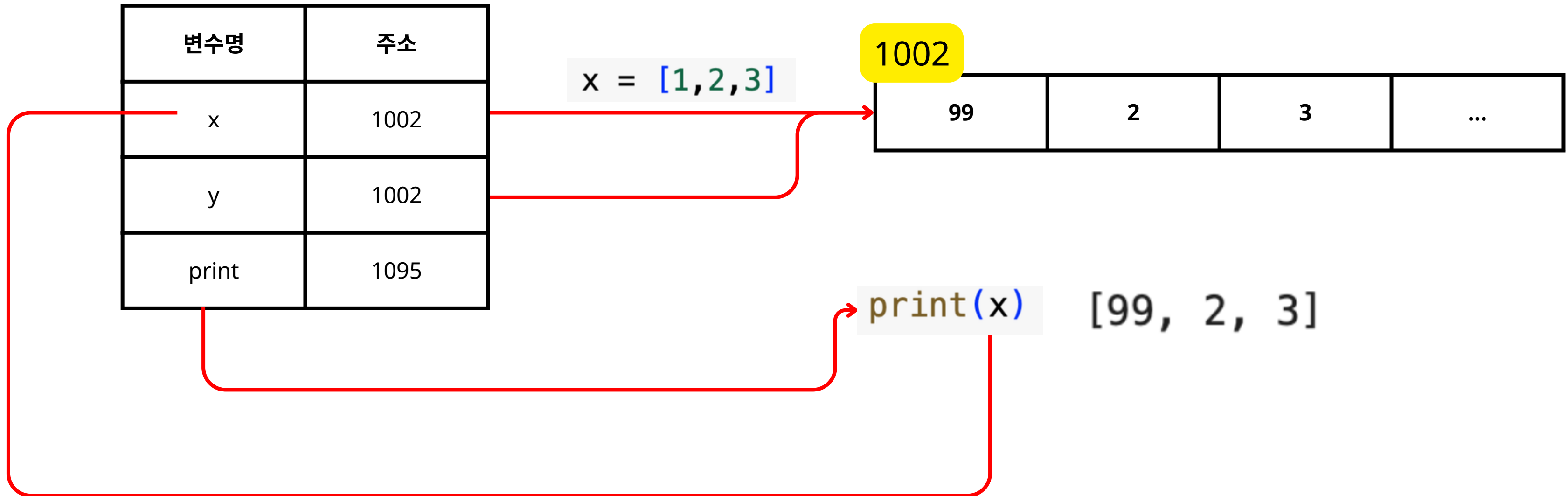
Call by reference

✓
0초



```
1 x = [1,2,3]
2 y = x
3 y[0] = 99
4 print(x)
```

⇒ [99, 2, 3]



List

mutable한 객체의 복사

- Shallow copy (얕은 복사)
 - `copy.copy()`, `list.copy()`
- Deep copy (깊은 복사)
 - `copy.deepcopy()`

가변객체의 참조 문제

```
# 가변 객체의 대입
a = [1, 2, 3]
b = a
b[0] = '*'
print(a)
```

`['*', 2, 3]`



복사!

```
1 x = [1,2,3]
2 y = x.copy()
3 y[0] = 99
4 print(x)
5 print(y)
```

`[1, 2, 3]`
`[99, 2, 3]`



얕은 복사의 문제점

```
import copy
a = [['a', 'b'], 1, 2, 3]
b = copy.copy(a)
# b = a.copy()
b[1] = '*'
print(a)
b[0][0] = '*'
print(a)
```

`[['a', 'b'], 1, 2, 3]`
 `[['*', 'b'], 1, 2, 3]`



```
# 깊은 복사
a = [['a', 'b'], 1, 2, 3]
b = copy.deepcopy(a)
b[0][0] = '*'
print(a)
```

`[['a', 'b'], 1, 2, 3]`

같은 메모리 주소를 참조한 것이지,
객체를 복사한 것이 아님

객체를 반복적으로 탐색해서
모두 복사해버림

iterable? iterator?

`__iter__`, `__next__`, `__getitem__`

`__iter__`가 있으면 iterable!

```
[43] 1 dir(my_list)
```

```
['__add__',  
 '__class__',  
 '__class_getitem__',  
 '__contains__',  
 '__delattr__',  
 '__delitem__',  
 '__dir__',  
 '__doc__',  
 '__eq__',  
 '__format__',  
 '__ge__',  
 '__getattr__',  
 '__getitem__',  
 '__getstate__',  
 '__gt__',  
 '__hash__',  
 '__iadd__',  
 '__imul__',  
 '__init__',  
 '__init_subclass__',  
 '__iter__',  
 '__le__',
```

`__iter__`를 이용하여 Iterator를 만들어줌.

Iterator는 `__next__`를 가지고, `next()`로 순차적으로 값을 줌.

```
1 my_list = [1,2,3,4]  
2 my_iterator = my_list.__iter__()  
3  
4 while True:  
5     print(next(my_iterator))
```

더이상 뱉을것이 없으면 에러!

```
1  
2  
3  
4
```

```
-----  
StopIteration                                Traceback (most recent call last)  
<ipython-input-41-a990ed9b31d0> in <cell line: 0>()  
      3
```

```
      4 while True:  
----> 5     print(next(my_iterator))
```

```
StopIteration:
```

값을 다 주고나면 더 이상 동작하지 않음. (1회성)

iterable? iterator?

`__iter__`, `__next__`, `__getitem__`

`__iter__`가 있으면 무조건 iterable?
○○○ 그러함

`__iter__`가 없어도 iterable인 경우는?
`__getitem__` 정의시 알아서 iterable로 변환해서 처리해줌.

itertools 라이브러리에는 다양한 iterator를 제공한다

이후 generator와 iterator의 차이도 알아보면 좋다.

```
1 from typing import Iterable |
2
3 class my_iterable:
4     def __init__(self):
5         pass
6
7     def __iter__(self):
8         pass
9
10 mi = my_iterable()
11 print(isinstance(mi, Iterable))
```

True

`__iter__`있으면 Iterable하다고 판단.

```
1 class MySequence:
2     def __getitem__(self, index):
3         if index < 5:
4             return index * 10
5         else:
6             raise IndexError("Index out of range")
7
8 seq = MySequence()
9 for item in seq:
10     print(item)
```

0
10
20
30
40

파이썬은 `__iter__()`가 없으면 `__getitem__(i)`을 $i=0,1,2,\dots$ 순서로 호출하며, `IndexError`가 발생할 때까지 반복하는 "레거시 iterable" 처리를 지원합니다.

정리

- Tuple은 '보관', List는 '관리'
- 리스트 슬라이싱
- 메소드 이것저것 해보기
- 빈칸 채워오기!

Dictionary

Key - value 모음!
{“이름”:이창민, “학년”:15}

hashable: `__hash__`

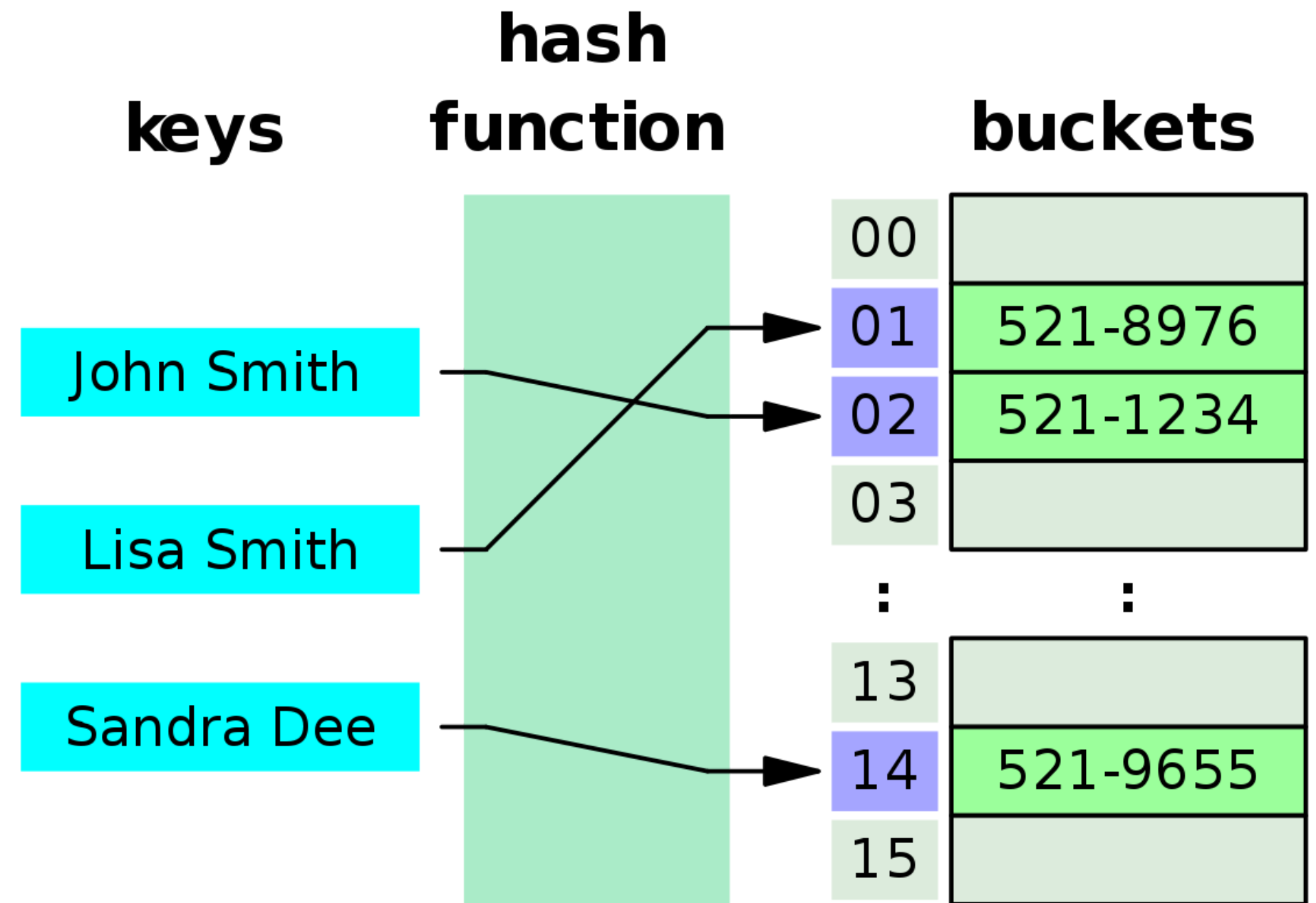
키와 불변객체

```
✓ [71] 1 dir("")
0초
↳ ['__hash__', '__init__', ...]

✓ 1 dir(1)
0초
↳ ['__get__', '__getattr__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', ...]
```

what is
the hash?

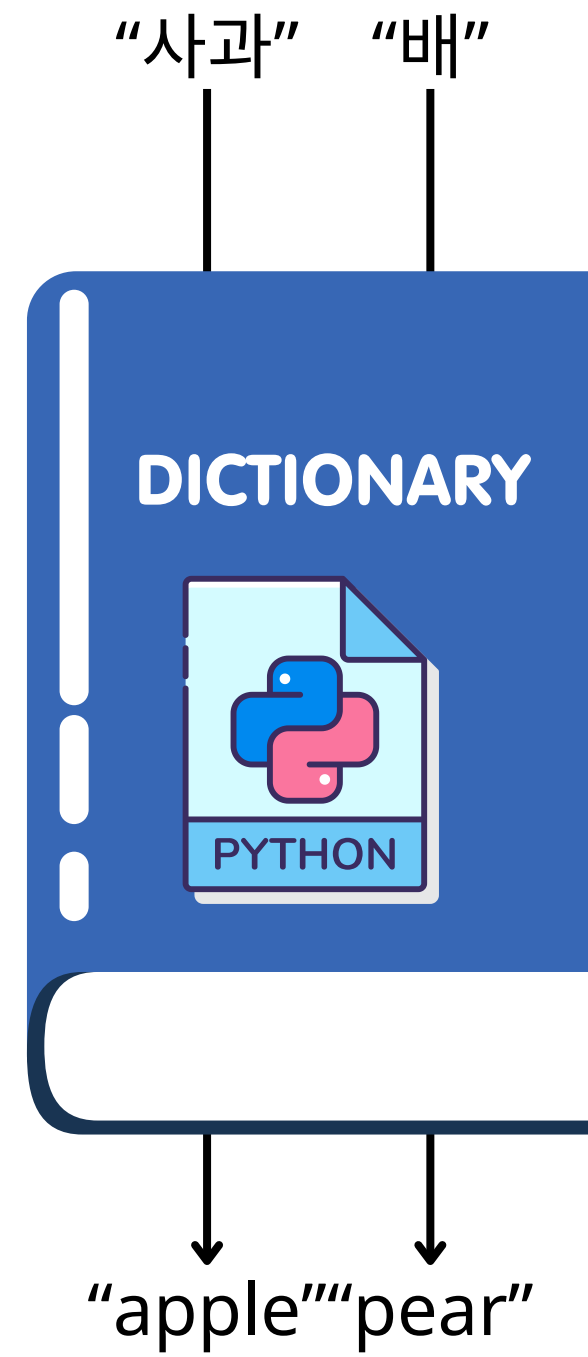
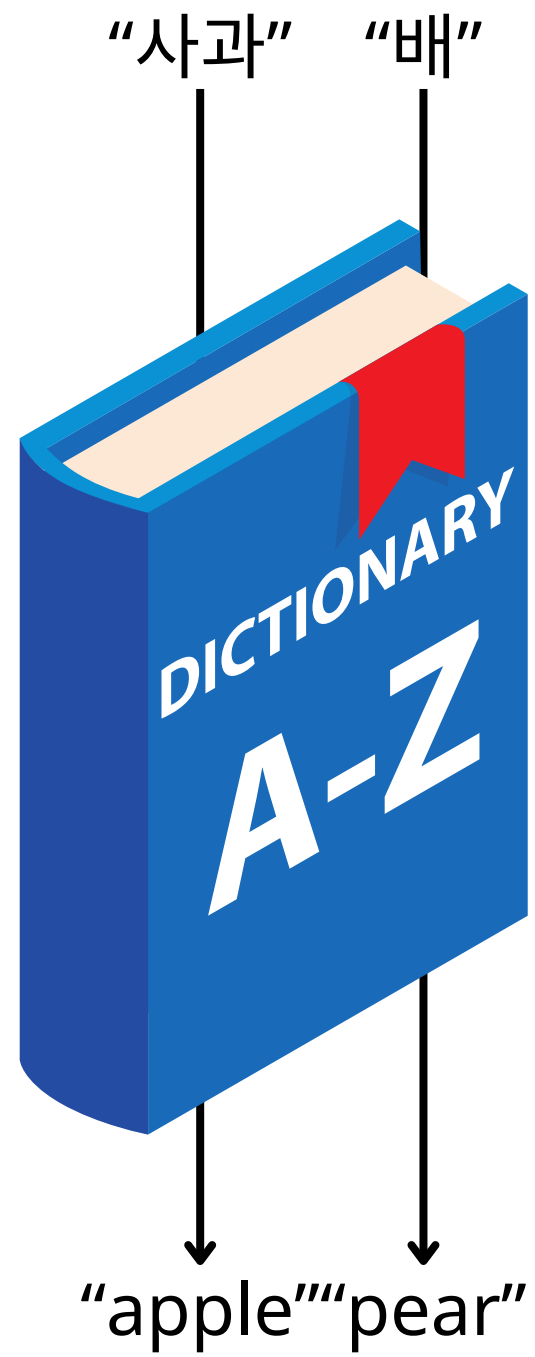
`__hash__`가 있다는건?
`hash()`에 넣을 수 있다는것!
= hashable하다는 것!



key → Value (숫자=주소)
Hash Function: key를 value로 바꿔주는 함수

어떤 입력을 주소로 바꿔주는 것

다른 길이, 다른 형태이더라도..



```
✓ 0초
▶ 1 d={"사과": "apple",
2   "배": "pear",
3   "귤": "mandarin",
4   }
5 print(d["사과"])
6 print(d["배"])
```

⇌ apple
pear

Dictionary

특징

- 선언

```
d = {}  
d['a'] = 1  
d['b'] = 2  
d
```

→ {'a': 1, 'b': 2}

{ } 를 기호로 사용
key:value 형태로 나열
, 로 구분

```
d = {}  
type(d)
```

dict

```
d = dict()  
type(d)
```

dict

```
{'a' : 1, 'a' : 2}
```

```
{'a' : 2}
```

같은 건 마지막 거로

키가 중복되면 덮어쓰기

- 함수로 선언 vs {}로 선언

```
[10] 1 d = dict(alice=5, bob=20, tony=10)    #dict()함수이용  
      2 d = {"alice":5, "bob":20, "tony":10}  #{}기호 이용  
      3 print(d)
```

↪ {'alice': 5, 'bob': 20, 'tony': 10}

- 타입캐스트 : key, value 짝만 맞으면 set, list 다 가능

```
name_ages1 = dict([['alice', 5], ['Bob', 13]])  {'alice': 5, 'Bob': 13}  
name_ages2 = dict([('alice', 5), ('Bob', 13)])  {'alice': 5, 'Bob': 13}  
name_ages3 = dict(('alice', 5), ('Bob', 13))    {'alice': 5, 'Bob': 13}  
name_ages4 = dict(['alice', 5], ['Bob', 13])    {'alice': 5, 'Bob': 13}
```

Dictionary

값 바꾸기 등

- 값 바꾸기

```
✓ [17] 1 a = {'alice': [1, 2, 3], 'bob': 20, 'tony': 15}
0초    2 a['cm']=88
       3 print(a)
       4 a['alice']=0
       5 print(a)
```

현재 없던 key:value가 들어오면 dict에 추가

존재하는 key:value가 들어오면 dict update

```
⇒ {'alice': [1, 2, 3], 'bob': 20, 'tony': 15, 'cm': 88}
   {'alice': 0, 'bob': 20, 'tony': 15, 'cm': 88}
```

- update
: key 있으면 update, 없으면 key-val 추가

```
a = {'alice': [1, 2, 3], 'bob': 20, 'tony': 15}
a.update({'bob': 99, 'kim': 30})
a
```

```
{'alice': [1, 2, 3], 'bob': 99, 'kim': 30, 'tony': 15}
```

- len(): __len__, str(): __str__

```
✓ [15] 1 print(len(a)) = 키 갯수
0초    2 print(str(a), str(a)[0:3])
```

```
⇒ 3
   {'alice': [1, 2, 3], 'bob': 20, 'tony': 15} {'a
```

- fromkeys(): 여러 키값으로 딕셔너리 생성

```
# fromkeys : iterable 키 값으로 사전 생성
dict.fromkeys(['one', 'two', 'three'], 10)

{'one': 10, 'three': 10, 'two': 10}
```

Dictionary

key는 불변객체만 가능하다!

- key 자료형

```
data = {}  
data[(1,2)] = {1,2}
```

```
data[{1,2}] = (1,2)
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-37-d806c34c2b45> in <module>()  
----> 1 data[{1,2}] = (1,2)
```

TypeError: unhashable type: 'set' # key는 hashable = __hash__ = 불변객체!

왜??

- value 자료형

```
{1:2, '참민':False, True:[1,2]}
```

다양한 자료형 가능

```
{1: [1, 2], '참민': False}
```

Dictionary

인덱스

- Index 접근 안됨. 순서가 없는 자료형

```
d = {'abc' : 1, 'def' : 2}
```

```
d[0]
```

0이란 key는 없어서 KeyError 인 점 주목

```
-----  
KeyError                                Traceback (most recent call last)  
<ipython-input-16-123a9cc6df61> in <module>()  
----> 1 d[0]
```

- 키로는 접근 됨

```
d['abc']
```

```
1
```

```
d['abc'] = 5
```

```
d
```

```
{'abc': 5, 'def': 2}
```

```
d['ghi'] = 999
```

```
d
```

```
{'abc': 5, 'def': 2, 'ghi': 999}
```

key 없으면 key-value 추가

Dictionary

keys(), values(), items()

✓
0초

```
1 a = {'alice': [1, 2, 3], 'bob': 20, 'tony': 15}
2 print(a.keys(), type(a.keys()), list(a.keys()))
3 print(a.values(), type(a.values()), list(a.values()))
4 print(a.items(), type(a.items()), list(a.items()))
```

```
dict_keys(['alice', 'bob', 'tony']) <class 'dict_keys'> ['alice', 'bob', 'tony']
dict_values([[1, 2, 3], 20, 15]) <class 'dict_values'> [[1, 2, 3], 20, 15]
dict_items([('alice', [1, 2, 3]), ('bob', 20), ('tony', 15)]) <class 'dict_items'> [('alice', [1, 2, 3]), ('bob', 20), ('tony', 15)]
```

- Key, value 체크

```
a = {'alice': [1, 2, 3], 'bob': 20, 'tony': 15}
```

```
'alice' in a      20 in a      20 in a.values()
```

True

False

True

- 반복문에서의 활용

```
for key in a:      for val in a.values():      for key, val in a.items():
    print(key)      print(val)                  print(f'key = {key}, value = {val}')
```

alice

[1, 2, 3]

key = alice, value = [1, 2, 3]

bob

20

key = bob, value = 20

tony

15

key = tony, value = 15

value로는 검색 안됨 -----하지만 요렇게 되지

key, value 둘 다 반환

Dictionary

메소드 1

- 키를 가지고 삭제도 가능

```
a = {'alice': [1,2,3], 'bob': 20, 'tony': 15}
del a['alice']
```

a

{'bob': 20, 'tony': 15}

- clear() : dict 초기화

```
d = {'a': 1, 'b': 2}
print(d.clear())
```

None

- pop(), popitem()

```
d = {1: 'apple', 2: 'banana', 3: 'egg'}
```

```
print(d.pop(1))
print(d)
```

apple

{2: 'banana', 3: 'egg'}

```
print(d.popitem())
print(d)
```

(3, 'egg')

{1: 'apple', 2: 'banana'}

Dictionary

메소드 2

- get : key 있으면 value, 없으면 None

```
d = {'a': 1, 'b': 2}
print(d.get('a'))
print(d.get('c'))
print(d.get('c', '그런거 없어'))
```

```
1
None
그런거 없어
```

두번째 인자 있으면 None 대신 반환

Setdefault() : 해당 키가 없을 경우, 새로운 키와 지정한 값을 딕셔너리에 넣어서 반환

- 정렬

```
d = {'c': 2, 'a': 3, 'b': 1}
```

```
sorted(d)
```

['a', 'b', 'c'] # 그냥 쓰면 리스트로 반환

```
sorted(d.items())
```

```
[('a', 3), ('b', 1), ('c', 2)]
```

```
d = {'a': 1, 'b': 2}
```

```
print(d.setdefault('c', 4))
```

```
print(d)
```

4

{'a': 1, 'b': 2, 'c': 4}

Dictionary

- 중복이 불가능한 collection 자료형
- mutable(가변성), key는 불변객체
- key-value 구조
- key 중복이 안됨

```
d = dict()           // 딕셔너리 선언
d.keys()             // 딕셔너리의 키 목록
d.values()           // 딕셔너리의 값 목록
d.items()            // 딕셔너리의 key-value 튜플 목록
d.clear()            // 딕셔너리의 모든 요소 삭제
d.copy()             // 딕셔너리 복사
d.fromkeys()         // seq, value 셋으로 딕셔너리 생성
d.get(key, default=None) // 해당 키의 저장된 값 확인
d.setdefault(key, default=None) // 해당 키의 저장된 값 확인(만약 값이 없으면 None)
d.update(d2)         // 딕셔너리에 'd2'라는 딕셔너리를 추가
len(d)              // 딕셔너리 길이
str(d)              // 딕셔너리를 문자열로 변환
```

Dictionary

mutable한 객체의 복사

- Shallow copy (얕은 복사)
 - 인덱스, =, copy.copy(), list.copy()
- Deep copy (깊은 복사)
 - copy.deepcopy()

```
d = {'a': [1,2,3], 'b': 2}
shallow = copy.copy(d)

shallow['a'][0] = '*'

print(d, hex(id(d)))
print(shallow, hex(id(shallow)))
```

```
{'a': ['*', 2, 3], 'b': 2} 0x7f925b72a6e0
{'a': ['*', 2, 3], 'b': 2} 0x7f925b72a730
```

```
import copy
d = {'a': [1,2,3], 'b': 2}
deep = copy.deepcopy(d)

d['a'] = [4,5]
d['b'] = 20

print(d, hex(id(d)))
print(deep, hex(id(deep)))
```

```
{'a': [4, 5], 'b': 20} 0x7fa3b4c6e780
{'a': [1, 2, 3], 'b': 2} 0x7fa3b4c6b730
```

Dictionary

키와 hash?!

- hash() 함수의 동작!

```
0초 ▶ 1 print(hash(0))
      2 print(hash(0.0))
      3 print(hash(False))
      4 print(hash(""))
      5 print(hash(None))
      6 print(hash(tuple()))
```

```
⇒ 0
   0
   0
   0
   605712
   5740354900026072187
```

0, 0.0, False, ""는 같은 hash값!

None하고 tuple()은 따로 처리하네?

- 같은 주소에 매칭?

```
0초 ▶ 1 d={}
      2 d[0]=1
      3 print(d)
      4 d[0.0]=2
      5 print(d)
      6 d[False]=3
      7 print(d)
      8 d[""]=4
      9 print(d)
     10 d[None]=5
     11 d[tuple()]=6
     12 print(d)
```

```
⇒ {0: 1}
   {0: 2}
   {0: 3}
   {0: 3, '': 4}
   {0: 3, '': 4, None: 5, (): 6}
```

"" ?????

- 중복되는 키는 처음쓴걸 따라가는듯?

```
0초 [25] ▶ 1 d={}
          2 d[False]=1
          3 print(d)
          4 d[0.0]=2
          5 print(d)
          6 d[0]=3
          7 print(d)
```

```
⇒ {False: 1}
   {False: 2}
   {False: 3}
```

먼저 등록된 키를 따라가네

빈칸 채우기

Dictionary 예제

- Dictionary 선언
- 가변객체의 복사
- Dictionary 메소드

(1)

```
# Tuple List로부터 dict 생성
ages = [('철수', 30), ('민수', 35), ('영희', 25)]
mydict = dict(ages)
mydict
```

(2)

```
mydict = {'a': 1, 'b': 2}
for i in mydict:
    print(i, end = ' ')
```

(3)

```
menuA = {'떡볶이': 500, '오뎅': 300}
menuB = menuA
menuA['떡볶이'] = 1000
print(menuB)
```

(4)

```
scores = {'철수': 90, '민수': 85, '영희': 80}
for name, score in scores.items():
    print(f"{name}의 점수 : {score}점")
```

(5)

```
d = {4 : 'a', 2 : 'b', 7 : 'c'}
sorted(d)
```

OrderedDict?

dictionary는 순서가 없다?

```
>>> dic = {}
>>> dic['A'] = 1
>>> dic['B'] = 2
>>> dic['C'] = 3
>>> dic
{'A': 1, 'C': 3, 'B': 2}
>>> for key, val in dic.items():
...     print(key, val)
...
A 1
C 3
B 2
```

파이썬 3.6 이전
딕셔너리 순서가
마구잡이~

```
>>> from collections import OrderedDict
>>> ordered_dic = OrderedDict()
>>> ordered_dic['A'] = 1
>>> ordered_dic['B'] = 2
>>> ordered_dic['C'] = 3
>>> ordered_dic
OrderedDict([('A', 1), ('B', 2), ('C', 3)])
>>> for key, val in ordered_dic.items():
...     print(key, val)
...
A 1
B 2
C 3
>>> isinstance(ordered_dic, dict)
True
```

그럴때 collections의 다양한 자료형 중
OrderedDict가 순서를 가진 딕셔너리~

✓
0초

```
[45] 1 d = {}
      2 d["A"]=1
      3 d["B"]=2
      4 d["C"]=3
      5 for k,v in d.items():
      6     print(k,v)
```

⇒ A 1
B 2
C 3

지금은?

순서가 지켜지지롱

OrderedDict는 왜 남아있지?

dictionary의 대소비교는 이상하다

```
[46] 1 # 그럼 OrderedDict는 필요없나?  
2 d1 = {'A': 1, 'B': 2, 'C': 3}  
3 d2 = {'A': 1, 'C': 3, 'B': 2}  
4 print(d1 == d2)
```

키:값이 같다면 순서를 무시해도 같다고 표출

⇒ True

✓
0초

```
1 from collections import OrderedDict  
2 ordered_d1 = OrderedDict({'A': 1, 'B': 2, 'C': 3})  
3 ordered_d2 = OrderedDict({'A': 1, 'C': 3, 'B': 2})  
4 print(ordered_d1 == ordered_d2)
```

⇒ False

같음을 비교할때 순서 또한 중요하다면
OrderedDict

setdefault vs defaultdict

```
def count_letters(word):
    counter = {}
    for letter in word:
        counter.setdefault(letter, 0)
        counter[letter] += 1
    return counter

count_letters("banana") # {'b': 1, 'a': 3, 'n': 2}
```

dict.setdefault(new_key, default_value)
if new_key in dict:
 dict[new_key] = default_value

매번 setdefault를 하는것도 불편한데.. > defaultdict

```
[6] 1 from collections import defaultdict
      2
      3 def count_letters(word):
      4     counter = defaultdict(int)
      5     for letter in word:
      6         counter[letter] += 1
      7     return counter
      8
      9 count_letters('apple')
```

defaultdict(int, {'a': 1, 'p': 2, 'l': 1, 'e': 1})

```
1 help(defaultdict)
```

Help on class defaultdict in module collections:

```
class defaultdict(builtins.dict)
|   defaultdict(default_factory=None, /, [...]) --> dict with default factory
|
|   The default factory is called without arguments to produce
|   a new value when a key is not present, in __getitem__ only.
|   A defaultdict compares equal to a dict with the same items.
|   All remaining arguments are treated the same as if they were
|   passed to the dict constructor, including keyword arguments.
```

Set

집합!
중복원소는 허용되지 않는다.

Set

특징

- 선언

```
s1 = set([1,2,3])  
s1
```

{1, 2, 3}

```
s4 = {1,2,3}  
s4
```

{1, 2, 3}

```
s2 = set("Jaehun")  
s2
```

{'J', 'a', 'e', 'h', 'n', 'u'}

```
s3 = set("Hello")  
s3
```

{'H', 'e', 'l', 'o'}

```
s3t = {1,1,1,1,1,1,1,1,1}  
s3t
```

{1}

중복된 원소는
삭제된다.

- 선언 시 주의할 점

```
s5 = {1,2,3,4,5}  
s6 = {}  
print(type(s5))  
print(type(s6))
```

<class 'set'>

<class 'dict'>

빈 대괄호는 dictionary로 정의된다.
원소가 있는 대괄호는 set!

Set

특징

- 선언 시 주의할 점

```
s = {"1", 3, 5, [1, 3]}
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-16-d63171d7910a> in <module>()  
----> 1 s = {"1", 3, 5, [1, 3]}
```

TypeError: unhashable type: 'list'

```
s = {"1", 3, 5, {1, 3}}
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-17-2d67e0058e83> in <module>()  
----> 1 s = {"1", 3, 5, {1, 3}}
```

TypeError: unhashable type: 'set'

```
s = {"1", 3, 5, {1:1, 2:3}}
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-18-2f07ac5b843e> in <module>()  
----> 1 s = {"1", 3, 5, {1:1, 2:3}}
```

TypeError: unhashable type: 'dict'

set은 원소로 불변객체만 받는다.
(Immutable)

Set

인덱스

- Index 접근 안됨. 순서가 없는 자료형

```
s7 = {1,2,'e','d','a','g','t',5,9}
```

```
print(s7)
```

```
{1, 2, 5, 'd', 't', 9, 'e', 'g', 'a'}
```

```
s7[4]
```

Dict랑은 다르게 TypeError인 점 주목

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-20-dbcec6eeef67> in <module>()  
----> 1 s7[4]
```

TypeError: 'set' object is not subscriptable

- Index 접근 시 리스트나 튜플로 변환하여 접근.

```
s7_t = tuple(s7)  
print(s7_t[4])
```

t

```
s7_l = list(s7)  
print(s7_l[4])
```

t

key가 없어서 key로도 접근 불가

“이렇게 불편한 친구를
왜 만들었나요?”

Set

“집합”적인 메소드

```
s_et = {1,2,3,4,5,6}
se_t = {4,5,6,7,8,9}
```

교집합 : & or .intersection()

<code>s_et & se_t</code>	<code>s_et.intersection(se_t)</code>
{4, 5, 6}	{4, 5, 6}

합집합 : | or .union()

<code>s_et se_t</code>	<code>s_et.union(se_t)</code>
{1, 2, 3, 4, 5, 6, 7, 8, 9}	

차집합 : - or .difference()

<code>s_et - se_t</code>	<code>s_et.difference(se_t)</code>
{1, 2, 3}	{1, 2, 3}

<code>se_t - s_et</code>	<code>se_t.difference(s_et)</code>
{7, 8, 9}	{7, 8, 9}

대칭 차집합 (합집합 - 교집합) : ^ or .symmetric_difference()

<code>s_et.symmetric_difference(se_t)</code>	<code>s_et ^ se_t</code>
{1, 2, 3, 7, 8, 9}	{1, 2, 3, 7, 8, 9}

Set

“집합”적인 메소드

- 공통 원소를 갖지 않는가?

```
{1, 2, 3, 4}.isdisjoint({3, 4, 5, 6})
```

False

```
{1, 2, 3, 4}.isdisjoint({7, 8, 9})
```

True

- 부분집합인가?

```
{3, 4}.issubset({2, 3, 4, 5})
```

True

```
{1, 4, 7}.issubset({2, 3, 4, 5})
```

False

- 확대집합인가?

```
{2, 3, 4, 5}.issuperset({3, 4})
```

True

```
{2, 3, 4, 5}.issuperset({1, 4, 7})
```

False

is로 시작하는 함수는 True 또는 False를 리턴한다

Set

기타 메소드

```
s = {1, 2, 3, 4, 5, 6, 7}
```

- 새로운 값 추가(중복 값은 무시)

```
s.add(10)  
s
```

```
{1, 2, 3, 4, 5, 6, 7, 10}
```

- 임의의 값을 리턴하고 삭제

```
print(s.pop())  
print(s)
```

```
1  
{2, 3, 4, 5, 6, 7}
```

- 값 삭제

```
s.discard(4)  
s
```

```
{1, 2, 3, 5, 6, 7}
```

```
s.discard(100)  
s
```

```
{1, 2, 3, 5, 6, 7}
```

```
s.remove(7)  
s
```

```
{1, 2, 3, 4, 5, 6}
```

```
s.remove(100)  
s
```

```
-----  
KeyError                                Traceback (most recent call last)  
<ipython-input-25-3f9aa703733d> in <module>()  
----> 1 s.remove(100)  
      2 s
```

```
KeyError: 100
```

Set

정리

- 요소의 중복이 불가능한 내장모듈 collection 자료형
- mutable(가변성)
- Not iterable
- 중복제거, 교집합, 합집합, 차집합 등의 수학적 계산이 가능
- add(), update(), remove() 메소드를 활용하여 요소 추가/삭제 가능

```
s1 = {1, 2, 3, 4, 5}           // s1셋 선언
s2 = {3, 5, 7, 9}             // s2셋 선언
s = set([1,2,2,3,3,3,4,4,4,4]) // 셋으로 변환
s.add(5)                       // 요소 추가
s.update([6,7,8])              // 여러개 요소 추가
s.remove(8)                    // 요소 삭제
s1 | s2                        // s1 s2 합집합
s1 & s2                         // s1 s2 교집합
s1 - s2                        // s1 s2 차집합
s1 ^ s2                        // s1 s2 대칭 차집합
```

Set 예제

<pre>s1 = {1, 2, 3, 4, 5} s2 = {4, 5, 6, 7, 8}</pre>	<pre>s1 = {1, 2, 3, 4, 5} s2 = {4, 5, 6, 7, 8}</pre>	<pre>s1 = {1, 2, 3, 4, 5} s2 = {4, 5, 6, 7, 8}</pre>	<pre>s1 = {1, 2, 3, 4, 5} s2 = {4, 5, 6, 7, 8}</pre>
<pre># 합집합을 만들어서 리턴 s1.union(s2)</pre>	<pre># 차집합을 만들어 리턴 print(s1.difference(s2)) print(s1)</pre>	<pre># 교집합을 만들어 리턴 print(s1.intersection(s2)) print(s1)</pre>	<pre># 대칭차를 만들어 리턴 print(s1.symmetric_difference(s2)) print(s1)</pre>
<pre>{1, 2, 3, 4, 5, 6, 7, 8}</pre>	<pre>{1, 2, 3} {1, 2, 3, 4, 5}</pre>	<pre>{4, 5} {1, 2, 3, 4, 5}</pre>	<pre>{1, 2, 3, 6, 7, 8} {1, 2, 3, 4, 5}</pre>
<pre># 합집합을 만들어서 원본 데이터를 갱신 s1.update(s2) s1</pre>	<pre># 차집합을 만들어 원본 데이터를 갱신 s1.difference_update(s2) s1</pre>	<pre># 교집합을 만들어 원본 데이터를 갱신 s1.intersection_update(s2) s1</pre>	<pre># 대칭차를 만들어 원본 데이터를 갱신 s1.symmetric_difference_update(s2) s1</pre>
<pre>{1, 2, 3, 4, 5, 6, 7, 8}</pre>	<pre>{1, 2, 3}</pre>	<pre>{4, 5}</pre>	<pre>{1, 2, 3, 6, 7, 8}</pre>

Set 예제

```
# 리스트에서 중복값 제거하기
a = [1, 1, 2, 3, 3, 3, 4, 5]
a = list(set(a))
print(a)
```

[1, 2, 3, 4, 5]

```
# 튜플에서 중복값 제거하기
a = (1, 1, 2, 3, 3, 3, 4, 5)
a = tuple(set(a))
print(a)
```

(1, 2, 3, 4, 5)

```
myset = {1, 3, 5}
```

```
# 하나만 추가
myset.add(7)
myset
```

{1, 3, 5, 7}

```
# 여러 개 추가
myset.update({4, 2, 10})
myset
```

{1, 2, 3, 4, 5, 7, 10}

```
myset = {1, 3, 5}
```

```
# 하나만 삭제
myset.remove(1)
myset
```

{3, 5}

```
# 모두 삭제
myset.clear()
myset
```

set()

```
# 집합 연산자
a = {1, 3, 5}
b = {1, 2, 5}
```

```
# 교집합
i = a & b
i
```

{1, 5}

```
# 합집합
u = a | b
u
```

{1, 2, 3, 5}

```
# 차집합
d = a - b
d
```

{3}

빈칸 채우기

Set 예제

- Set 선언
- Set의 특징
- 집합 연산

(1)

```
s = set('Hello')  
print(len(s))
```

(2)

```
s = {}  
type(s)
```

(3)

```
{1, 5, 2, 6} + {6, 2, 7, 8}  
  
{1, 2, 5, 6, 7, 8}
```

(4)

다음 세 집합의 교집합을 구해보세요

```
set1 = {1, 2, 3, 6, 8}  
set2 = {2, 3, 4, 9, 6}  
set3 = {1, 2, 5, 4, 6}
```

```
print( )
```

{2, 6}

(5)

```
setA = {1, 2, 4, 7}  
setB = {2, 3, 7, 9}  
print( )
```

{9, 3}

(6)

다음은 A시험을 통과한 학생들과 B시험을 통과한 학생들의 명단이다.
둘 다 통과한 사람들을 출력해보자.

```
passA = {'민영', '동윤', '혜미', '소정', '재훈'}  
passB = {'재훈', '지수', '민기', '혜미'}  
print( )
```

FrozenSet?

```
[54] 1 frozen_s = frozenset([1, 2, 3, 4])
      2 frozen_s.add("a") #불가능
      3 frozen_s.discard("a") #불가능
      4 frozen_s.update({"a", "b", "c"}) #불가능
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-54-95c1ac3aee4d> in <cell line: 0>()
      1 frozen_s = frozenset([1, 2, 3, 4])
----> 2 frozen_s.add("a") #불가능
      3 frozen_s.discard("a") #불가능
      4 frozen_s.update({"a", "b", "c"}) #불가능

AttributeError: 'frozenset' object has no attribute 'add'
```

frozenset은 set과 동일. but **immutable**

```
1 s = {1, 2, 3, 4}
2 s.add("a")
3 print(s)
4 s.discard("a")
5 print(s)
6 s.update({"a", "b", "c"})
7 print(s)
```

```
{1, 2, 3, 4, 'a'}
{1, 2, 3, 4}
{1, 2, 3, 4, 'b', 'a', 'c'}
```

```
1 dir(frozen_s)

['__and__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getstate__',
 '__gt__',
 '__hash__',
```

```
[56] 1 d = {frozen_s: "frozenset"}
      2 print(d)

{frozenset({1, 2, 3, 4}): 'frozenset'}
```

키로도 쓸수있지!

```
1 s = frozenset((0, 1, 2, 3)) | set((2, 3, 4, 5))
2 print(type(s))
3 s = set((0, 1, 2, 3)) | frozenset((2, 3, 4, 5))
4 print(type(s))
```

```
<class 'frozenset'>
<class 'set'>
```

set과 연산도 가능한데 앞쪽 자료형을 따름.

정리

- 가변객체의 복사
 - dict, set의 선언
 - 순서가 없는 자료형
 - 중복 x
-
- 빈칸 채워오기!